The Journal of Open Source Software

btcaaron: a Python toolkit for reproducible Taproot script-path and covenant experiments on Bitcoin

Aaron Zhang [1]

1 Independent developer, United States

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 01 November 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Bitcoin coins are locked by small programs called scripts. Since the Taproot upgrade of 2021 (Wuille, Nick, & Ruffing, 2020; Wuille, Nick, & Towns, 2020a, 2020b), a single coin can hide a whole tree of alternative spending conditions — a hash lock, a multisignature, a time lock — and reveal only the branch that is actually used. Researchers are now studying proposed extensions, such as covenants that restrict *how* a coin may be spent next, on a public test network called Bitcoin Inquisition (Bitcoin Inquisition contributors, 2026), where these proposals are already active.

Building such experiments by hand is error-prone. Every spend requires the script tree, a Merkle proof called the control block, a signature over a carefully assembled message, and a witness stack in exactly the right order. A single wrong byte produces a rejected transaction with little explanation. `btcaaron` is a Python library that lets a researcher state the intended conditions declaratively, build and sign the transaction, and inspect every intermediate value, so that an on-chain experiment can be rebuilt and re-checked by someone else.

Statement of need

Research on Bitcoin script and covenant proposals increasingly rests on transactions confirmed on a public network rather than on argument alone. For that evidence to be checkable, the construction must be reproducible: another person needs to regenerate the same scripts and signature messages from a short program and check that the published witnesses satisfy them. Its tests rebuild previously confirmed testnet transactions from the same inputs and outputs and verify that their non-witness serializations produce the original TXIDs. `btcaaron` targets three groups: protocol researchers who prototype covenant constructions on Bitcoin Inquisition signet, developers learning Taproot script-path spending, and educators who need worked examples that run against a real network.

The library provides (i) a declarative builder for Taproot trees (`.hashlock()`, `.multisig()`, `.timelock()`, `.checksig()`, `.custom()`), (ii) script-path and key-path spend builders that construct the control block and order witnesses automatically, (iii) direct access to the raw BIP-341 signature-message preimage, (iv) an independent implementation of the BIP-118 ANYPREVOUT signature message (Decker & Towns, 2017), (v) the BIP-446 template hash (Sanders et al., 2026), and (vi) script templates for opcodes active on Bitcoin Inquisition: `OP_CAT` (Heilman & Sabouri, 2023), `OP_CHECKSIGFROMSTACK` (Black & Rubin, 2024a), `OP_CHECKTEMPLATEVERIFY` (Rubin, 2020) and `OP_INTERNALKEY` (Black & Rubin, 2024b).

DRAFT

The Journal of Open Source Software

State of the field

Several mature Python libraries handle Bitcoin transactions. `python-bitcoin-utils` (Karasavvas, 2025) offers low-level construction of scripts and transactions, including Taproot, and is designed for learning; `python-bitcoinlib` (Todd & python-bitcoinlib contributors, 2023) provides core data structures; `BitcoinLib` (1200wd, 2026) focuses on wallets and persistence; and `embit` (Snigirev, 2024) targets descriptors, PSBT and hardware-wallet integration. In the releases we inspected (python-bitcoin-utils 0.7.3 and 0.8.8, python-bitcoinlib 0.12.2, BitcoinLib 0.7.10, embit 0.8.0) we found no implementation of the BIP-118 signature message, `OP_CHECKSIGFROMSTACK`, `OP_CHECKTEMPLATEVERIFY` or the BIP-446 template hash. Reference Python code for these messages does exist in Bitcoin Inquisition's functional test framework (`test/functional/test_framework/script.py`), but it is test scaffolding tied to a node build rather than an installable library; `btcaaron` uses it as an independent oracle in its own tests. Outside Python, Sapio (Rubin & Sapio contributors, 2020) is a Rust language for contracts built on `OP_CHECKTEMPLATEVERIFY`; it targets contract authoring rather than byte-level experiments with several proposals side by side.

`btcaaron` builds on `python-bitcoin-utils` for key handling, Schnorr signing, address encoding and transaction serialization rather than re-implementing them. What it implements itself is the layer covenant research needs: Taproot tree construction and control blocks (`tapmath`), declarative script-path spending, the BIP-341 signature-message preimage, the BIP-118 and BIP-446 hash algorithms, the Inquisition script templates, and PSBT v0/v2 handling. These components are experimental by nature — they track draft proposals and test-network software — which is why they live in a separate research-oriented package instead of being proposed to a general-purpose library.

Software design

The package separates building, spending and broadcasting. `TapTree` collects leaves and produces a `TaprootProgram` that knows its address, leaves and control blocks. A program's `.spend(label)` or `.keypath()` returns a `SpendBuilder` that takes inputs, outputs and unlocking data, signs, and assembles the witness; signature ordering for `OP_CHECKSIGADD` multisignatures and `nSequence` for relative time locks are set automatically. Network access (block explorers, a local node over RPC) is isolated in a `network` module so that construction and verification work offline.

For covenant work the key design choice is transparency. `sigmsg` returns the complete message that is tagged-hashed into a Taproot signature hash (the epoch byte, the BIP-341 `SigMsg` fields and, for script-path spends, the BIP-342 extension), so that experiments can split it into fields, feed it to `OP_CHECKSIGFROMSTACK`, or detect which sighash flag a signer used. `bip118` implements the ANYPREVOUT message (`Msg118/Ext118`) from the specification, and `templatehash` implements BIP-446, cross-checked against the Bitcoin Inquisition v29.4 implementation. `RawScript` and `tapmath` let users write arbitrary tapscript, including opcodes that the underlying library does not name, and still compute leaf hashes, Merkle paths and tweaked keys. Programs and transactions provide an `.explain()` method, and programs a `.visualize()` tree view, that show what was built and why, which supports teaching use.

The repository contains 244 automated tests, of which 5 interactive tests that broadcast real transactions are skipped by default. Correctness is checked against external references: the official BIP-341 test vectors (signature messages for every sighash type), 88 vectors generated with Bitcoin Inquisition's own test framework for the BIP-118 signature message (all six ANYPREVOUT hash types, with and without an annex) and the BIP-446 template hash, and previously confirmed testnet transactions, whose non-witness serializations and TXIDs the builders reproduce. Continuous integration runs the suite on Python 3.10–3.12. An API reference generated from the docstrings is in `docs/API.md`.

DRAFT

The Journal of Open Source Software

Research impact statement

btcaaron is the construction and verification layer for a series of on-chain covenant experiments on Bitcoin Inquisition signet. Its examples/ directory and companion repositories reproduce: output and input binding with OP_CAT and OP_CHECKSIGFROMSTACK, and an eltoo-style (Decker et al., 2018) state chain built from an OP_CHECKSIGFROMSTACK delegation ladder with OP_CHECKTEMPLATEVERIFY settlement (Zhang, 2026b); ANYPREVOUT signing from scratch with an on-chain rebinding proof (Zhang, 2026a); and a six-transaction eltoo state chain using ANYPREVOUT (Zhang, 2026c). Each package lists the confirmed transaction IDs, and examples/verify_onchain.py, without private keys or a local node, checks that every cited transaction is confirmed and recomputes its TXID from the non-witness serialization; the companion repositories sighash-prevout-binding, eltoo-apo-state-chain and braidpool-rca-covenant add claim-by-claim key-free verifiers. Outside contributors have extended the library (support for arbitrary leaf counts, duplicate-label checks) and reported cross-node test results for Bitcoin Core v30.2, Knots v29.3 and Libre v30.2 (issue #5); Christian Decker, a co-author of eltoo and BIP-118, responded to the ANYPREVOUT state-chain experiment on Delving Bitcoin.

AI usage disclosure

Generative AI coding assistants were used during development: Cursor (agent mode; the underlying model was not recorded) and Anthropic Claude via Claude Code (Claude Opus 4.6, 4.8 and 5.5). They were used for code generation and refactoring of library modules (including the Bitcoin Inquisition script templates and the BIP 118 and BIP 446 hash implementations), test scaffolding, documentation (README, INQUISITION.md, example READMEs, the generated API reference) and for drafting and copy-editing this paper. OpenAI ChatGPT (model version not recorded) was used to review drafts of this paper; the author checked each suggestion against the code before adopting it. For the API, the assistants proposed several alternative designs; the author evaluated them and decided the final architecture. The choice of experiments and the correctness criteria were the author's.

The author designed the architecture (the declarative TapTree / TaprootProgram / SpendBuilder split, the separation of construction from network access, and the exposure of signature-message preimages so that one toolkit can drive many different covenant experiments), defined every on-chain experiment, and validated the code against external ground truth rather than against AI output: the official BIP-341 test vectors, reference vectors generated with Bitcoin Inquisition's test framework, and transactions that were broadcast and confirmed on testnet and signet, whose TXIDs are reproduced by the test suite. The author reviewed, edited and takes responsibility for all AI-assisted code, documentation and text.

Acknowledgements

Early development of btcaaron was supported by a grant from OpenSats. The funder had no role in the software design, the experiments, the validation or the preparation of this paper. We thank the contributors to btcaaron and the maintainers of Bitcoin Inquisition and python-bitcoin-utils.

References

1200wd. (2026). *BitcoinLib* (Version 0.7.10). https://github.com/1200wd/bitcoinlib

Bitcoin Inquisition contributors. (2026). *Bitcoin Inquisition* (29.4-inq). https://github.com/bitcoin-inquisition/bitcoin

DRAFT

Black, B., & Rubin, J. (2024a). *BIP 348: CHECKSIGFROMSTACK*. https://github.com/bitcoin/bips/blob/master/bip-0348.md

Black, B., & Rubin, J. (2024b). *BIP 349: OP_INTERNALKEY*. https://github.com/bitcoin/bips/blob/master/bip-0349.md

Decker, C., Russell, R., & Osuntokun, O. (2018). *eltoo: A Simple Layer2 Protocol for Bitcoin*. https://blockstream.com/eltoo.pdf

Decker, C., & Towns, A. (2017). *BIP 118: SIGHASH_ANYPREVOUT for Taproot Scripts*. https://github.com/bitcoin/bips/blob/master/bip-0118.mediawiki

Heilman, E., & Sabouri, A. (2023). *BIP 347: OP_CAT in Tapscript*. https://github.com/bitcoin/bips/blob/master/bip-0347.mediawiki

Karasavvas, K. (2025). *python-bitcoin-utils* (Version 0.7.3). https://github.com/karask/python-bitcoin-utils

Rubin, J. (2020). *BIP 119: CHECKTEMPLATEVERIFY*. https://github.com/bitcoin/bips/blob/master/bip-0119.mediawiki

Rubin, J., & Sapio contributors. (2020). *Sapio: a Bitcoin programming language*. https://github.com/sapio-lang/sapio

Sanders, G., Poinsot, A., & Roose, S. (2026). *BIP 446: OP_TEMPLATEHASH*. https://github.com/bitcoin/bips/blob/master/bip-0446.md

Snigirev, S. (2024). *embit* (Version 0.8.0). https://github.com/diybitcoinhardware/embit

Todd, P., & python-bitcoinlib contributors. (2023). *python-bitcoinlib* (Version 0.12.2). https://github.com/petertodd/python-bitcoinlib

Wuille, P., Nick, J., & Ruffing, T. (2020). *BIP 340: Schnorr Signatures for secp256k1*. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki

Wuille, P., Nick, J., & Towns, A. (2020a). *BIP 341: Taproot: SegWit version 1 spending rules*. https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki

Wuille, P., Nick, J., & Towns, A. (2020b). *BIP 342: Validation of Taproot Scripts*. https://github.com/bitcoin/bips/blob/master/bip-0342.mediawiki

Zhang, A. (2026a). *BIP 118 signing from scratch + on-chain rebinding proof*. Delving Bitcoin. https://delvingbitcoin.org/t/bip-118-signing-from-scratch-on-chain-rebinding-proof/2411

Zhang, A. (2026b). *Capability Parity of CSFS+CAT with CTV and ANYPREVOUT for Common Covenant Patterns: On-Chain Evidence from Inquisition Signet*. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=7003839

Zhang, A. (2026c). *Eltoo state chain on Signet: three rounds, six transactions (APO + CTV)*. Delving Bitcoin. https://delvingbitcoin.org/t/eltoo-state-chain-on-signet-three-rounds-six-transactions-apo-ctv/2413

DRAFT